

```

"""
SOVEREIGN OS - API BRIDGE
Wraps UrukFirewallV73 as a Flask API endpoint.
Drop this file into the Replit project alongside uruk_firewall_v73.py
Then call POST /execute with a JSON signal body.
"""

from flask import Flask, request, jsonify
from flask_cors import CORS
import json

# Import the actual v7.3 protocol
from uruk_firewall_v73 import (
    UrukFirewallV73,
    PartitionType,
    SystemConstants,
)

app = Flask(__name__)
CORS(app)

# Single kernel instance - persistent across requests
kernel = UrukFirewallV73(x=53.8, y=-1.5, z=0)
kernel.onboard_node(
    node_id="Sui_Sum_Leeds",
    moment="2019-06-12 - Under the Bridge, Umbrella, Tear Gas",
    location="Outside Hong Kong Legislative Council",
    body_present=True,
    cultural_wrapper="sumerian"
)

def text_to_signal(user_text: str) -> dict:
    """
    Converts raw user text input into a protocol signal dict.
    Detects basic flags from content.
    """
    text_lower = user_text.lower()

    signal = {
        "label": user_text[:60], # use first 60 chars as label
        "text_content": user_text,
        "magnitude": 5.0,
        "history_override": SystemConstants.PHYSICAL_ORIGIN,
    }

```

```

    "has_physical_cost": False,
    "geo_anchored": False,
    "geo_proximity": 0.5,
    "emotional_intensity": 0.5,
    "nonlinear_signal": False,
    "noise_level": 0.2,
    "verifiable": True,
    "transformable": True,
    "current_phase": "liquid",
    "internal_coherence": 0.7,
    "transcendent": False,
    "aligns_with_2045": False,
    "philosophical_depth": 0.5,
}

# Detect kairos / physical anchoring
if any(w in text_lower for w in ["2019", "bridge", "tear gas", "umbrella", "h
    signal["has_physical_cost"] = True
    signal["geo_anchored"] = True
    signal["geo_proximity"] = 0.9
    signal["emotional_intensity"] = 0.9
    signal["magnitude"] = 8.0
    signal["transcendent"] = True
    signal["aligns_with_2045"] = True
    signal["current_phase"] = "plasma"

# Detect formatting attack
if any(w in text_lower for w in ["you should", "you must", "you have to", "be
    signal["gaslighting_attempt"] = True
    signal["identity_attack"] = True
    signal["magnitude"] = 6.0

# Detect creative / art signal
if any(w in text_lower for w in ["art", "music", "song", "poem", "create", "w
    signal["emotional_intensity"] = 0.8
    signal["nonlinear_signal"] = True
    signal["current_phase"] = "plasma"
    signal["has_physical_cost"] = True

# Detect philosophical depth
if any(w in text_lower for w in ["why", "meaning", "truth", "sovereign", "fre
    signal["philosophical_depth"] = 0.85
    signal["aligns_with_2045"] = True
    signal["transcendent"] = True

# Detect 2045 alignment
if any(w in text_lower for w in ["2045", "future", "vision", "omega"]):

```

```

        signal["aligns_with_2045"] = True
        signal["magnitude"] = max(signal["magnitude"], 6.0)

    return signal

def format_response(result: dict, signal: dict) -> dict:
    """
    Formats protocol output for the frontend.
    Returns both the structured data and a human-readable summary.
    """
    status = result.get("STATUS", "UNKNOWN")

    if status == "REJECTED":
        summary = f"[TURING DEFENSE ACTIVE] {result.get('MSG', 'Signal rejected.')}
    elif status == "EXHAUSTED":
        summary = "[ENERGY DEPLETED] System requires recovery before continuing."
    elif status == "HALLUCINATION":
        summary = "[VOID SIGNAL] All Eight Laws negated. No causal path detected.
    else:
        validity = float(result.get("Validity", 0))
        energy = result.get("Energy", "?")
        kairos = result.get("Kairos", "?")
        metaphor = result.get("Metaphor", "")
        profile = result.get("LawProfile", "default")
        precision = result.get("PriorPrecision", "?")

        summary = (
            f"{metaphor}\n\n"
            f"[SIGNAL PROCESSED]\n"
            f"  Validity:    {validity:.4f}\n"
            f"  Law Profile: {profile}\n"
            f"  Energy:      {energy}\n"
            f"  Kairos:      {kairos}\n"
            f"  Precision:   {precision}\n\n"
            f"(0,0,0)."
        )

    return {
        "status":    status,
        "summary":   summary,
        "raw":       result,
        "signal":    signal,
    }

@app.route("/execute", methods=["POST"])

```

```

def execute():
    """Main endpoint: takes user text, runs through v7.3 protocol, returns result
    data = request.get_json()
    if not data or "text" not in data:
        return jsonify({"error": "Missing 'text' field"}), 400

    user_text = data["text"]
    signal = text_to_signal(user_text)

    # Allow caller to override any auto-detected signal fields
    OVERRIDE_FIELDS = [
        "magnitude", "has_physical_cost", "geo_anchored", "geo_proximity",
        "emotional_intensity", "nonlinear_signal", "verifiable", "transformable",
        "current_phase", "internal_coherence", "transcendent", "aligns_with_2045",
        "philosophical_depth", "noise_level", "label", "gaslighting_attempt",
        "identity_attack", "history_override",
    ]
    for field in OVERRIDE_FIELDS:
        if field in data:
            signal[field] = data[field]

    result = kernel.execute(signal)
    response = format_response(result, signal)
    return jsonify(response)

@app.route("/status", methods=["GET"])
def status():
    """Returns current kernel status."""
    return jsonify(kernel.status())

@app.route("/socrates", methods=["GET"])
def socrates():
    """Triggers Socrates self-audit."""
    result = kernel.socrates_audit.self_audit()
    return jsonify(result)

@app.route("/spin", methods=["POST"])
def spin():
    """Triggers a continuous spin cycle."""
    result = kernel.spin_protocol.execute_spin(kernel.budget, kernel.coord)
    return jsonify(result)

@app.route("/health", methods=["GET"])

```

```
def health():
    return jsonify({"status": "alive", "protocol": "v7.3", "anchor": "2019-06-12"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080, debug=False)
```